Guide Crash Course IA

Du zéro à la salle de classe — ML, vibe coding, MCP, CLI, agents, théorie des jeux

- Niveau : débutant (aucune maths lourdes)
- ~60 concepts expliqués simplement + glossaire A–Z
- Plan d'étude de 4 semaines inclus

Ce qu'il y a dedans

Module 0	Carte du territoire
Module 1	IA / Machine Learning / Deep Learning
Module 2	Les modèles de langage (LLM)
Module 3	Vibe coding
Module 4	MCP — les connecteurs de l'IA
Module 5	CLI — la sacoche de survie
Module 6	Agents et agentic
Module 7	Théorie des jeux (2 minutes)
+	Sommaire, glossaire A–Z, ressources gratuites

Mis à jour octobre 2026 · source : guide-crash-course-ia.md

Sommaire

Module 0 — Carte du territoire 3
Module 1 — IA, Machine Learning, Deep Learning : les bases 4
Module 2 — Les modèles de langage (LLM) : ce qui fait ChatGPT 6
Module 3 — Vibe coding : programmer avec les Vibes 7
Module 4 — MCP (Model Context Protocol) : les connecteurs de l'IA 8
Module 5 —_CLI : la sacoche de survie en ligne de commande 9
Module 6 — Agents et "agentic" : comprendre quand l'IA s'aggoit 10
Module 7 — Théorie des jeux : l'IA qui strategic entre elle 12
Plan suggéré de 4 semaines (ordre + effort) 13
Glossaire (A–Z des termes que tu croises en cours) 14
Ressources gratuites (et débutants-friendly) 17

Astuce : dans ton lecteur PDF, Ctrl+F (Chercher) pour retrouver un terme en une seconde. Le glossaire commence à la page marquée « Glossaire ».

Pour qui ?
les agents
l'ordre la p
cours. Chac

Module 0 — Carte du territoire

```
 INTELLIGENCE ARTIFICIELLE (l'objectif : faire "raisonner" une machine)
│
├── MACHINE LEARNING (une méthode : apprendre depuis des données)
│   ├── Supervised (apprentissage supervisé)    → avec des réponses d'exemple
│   ├── Unsupervised (non supervisé)            → trouver des motifs tout seul
│   └── RL / Reinforcement (par essais)         → apprendre par essai-erreur + récompenses
│
├── DEEP LEARNING (une sous-méthode : réseaux de neurones profonds)
│   └── TRANSFORMERS → les modèles de langage (GPT, Claude, Llama…)
│
├── IA GÉNÉRATIVE (produire du neuf : texte, image, code, musique)
│
└── AGENTS / AGENTIC (des IA qui agissent, pas seulement qui répondent)
    └── MCP (comment les agents accèdent à des outils externes)
```

Ordre de lecture logique pour toi : Mod 1 (ML) Mod 2 (LLM) Mod 3 (Vibe coding) Mod 4 (MCP) Mod 5 (CLI) Mod 6 (Agents) Mod 7 (Théorie des jeux).

Module 1 — IA, Machine Learning, Deep Learning : les bases

1.1 Les trois niveaux

Niveau	Question	Analogie quotidienne
---	---	---
IA	"Peut-on faire faire à une machine ce qui demande de l'intelligence ?"	Le rêve
ML	"...en l'entraînant sur des exemples au lieu de tout coder à la main."	Le sport études
DL	"...avec des réseaux de neurones pour les problèmes très complexes."	L'équipe de haut niveau

Idée clé : avant le ML, on codait des règles à la main. Depuis le ML, on donne des exemples et la machine déduit les règles.

1.2 Les trois types d'apprentissage

- **Supervisé** : on donne des paires *entrée* *réponse attendue*. - Ex. : 10 000 photos étiquetées "chat"/"chien" le modèle apprend à distinguer. - Utilisé pour : classification, prédiction, traduction. - **Non supervisé** : on ne donne que les entrées ; le modèle cherche des groupes. - Ex. : regrouper les clients d'un site en profils sans qu'on lui dise lesquels. - Utilisé pour : clustering, recommandation d'anomalies. - **Par renforcement (RL)** : l'agent agit, gagne ou perd des points, apprend de l'expérience. - Ex. : aller d'un point A à un B, Jeu d'échecs, chatGPT (RLHF) — voir 1.5.

1.3 Comment on "apprend" (sans maths)

La intuition en 4 étapes :

- **Données** : des exemples d'entrée + la bonne réponse (jeu supervisé).
- **Prédiction** : le modèle devine une réponse.
- **Erreur (loss = la "note d'erreur")** : on mesure l'écart entre deviné et vérité. Plus l'écart est grand, plus la note est mauvaise.
- **Ajustement (backpropagation = rétropropagation)** : on remonte cette info et on ajuste *légèrement* toutes les "tourettes" (les poids) du réseau, puis on recommence.

Image mentale à garder : imagine que tu descends une colline dans le brouillard, pied nue. Tu tends le pied, tu sens si la pente monte ou descend, tu fais un pas dans le bon sens, et tu recommences des milliers de fois. C'est exactement ça, l'apprentissage : ajuster petit pas par petit pas jusqu'au point le plus bas (l'erreur minimale). Ce schéma s'appelle la **descente de gradient**. - On répète tout le jeu de données plusieurs **tours (époques / epochs)**. - Les nombres manipulés sont souvent rangés dans des **tenseurs** (des tableaux de nombres à 1, 2, 3... dimensions).

1.4 Neurones et réseaux (2 minutes)

- Un **neurone artificiel** est une mini-machine à calculer : numbers en entrée, un calcul simple, un résultat out. - Plusieurs neurones une **couche**. - Plusieurs couches empilées un **réseau de neurones**, et c'est pourquoi c'est *deep* (profond). - Les poids des neurones, c'est ce qu'on ajuste à l'entraînement. Un modèle comme GPT en contient des **dizaines de milliards**.

1.5 Sur-apprentissage vs sous-apprentissage

Problème	Image	Traitement

Problème	Image	Traitement
Overfitting	L'élève qui a appris son cours par cœur et rate dès que le prof change une tournure (obsolète).	Plus de données, plus simple, arrêter plus tôt
Underfitting	L'élève qui n'a rien compris de la matière.	Modèle plus complexe, plus d'entraînement

💬 Tu rencontreras aussi le terme **RLHF** (Reinforcement Learning from Human Feedback) — c'est l'entraînement final d'un chatbot : des humains notent les réponses, et le modèle apprend à mieux répondre, puis ça devient plus cohérent.

À retenir : (1) ML = apprendre des exemples, pas coder des règles ; (2) apprdissage = boucle prédiction erreur ajustement ; (3) overfitting = trop de mémoire, underfitting = pas assez de compréhension.

Module 2 — Les modèles de langage (LLM) : ce qui fait ChatGPT

2.1 Le transformer : l'architecture qui a tout changé

- Avant : des réseaux qui lisaient les mots **un par un** dans l'ordre (comme une story). - maintenant : **le transformer** lit tout le texte d'un coup et, pour chaque mot, calcule à **quelle mesure les autres mots du texte "méritent son attention"**. - **Attention** = poids : dans "Le chat s'est assis sur la tapis, il était confortable", pour comprendre "il", le modèle donne beaucoup d'attention à "chat" et presque aucune au tapis.

> **Image mentale :** imagine chaque mot de ta phrase qui envoie des fils plus ou moins forts vers les autres mots du texte. L'épaisseur du fil = son importance pour comprendre celui-ci. C'est exactement ce que fait le mécanisme d'attention.

2.2 Les tokens

- Un grand modèle ne comprend pas les mots, il comprend des **tokens** : des **morceaux** de mots. 100 tokens ≈ 75 mots en français. - Tout ce que tu envoies (prompt) est converti en tokens, et compté en tokens (avantage : les prix des APIs se font **au token**). - Un token n'est pas un mot : "impossible" pourrait devenir "im" + "possible".

2.3 La vie d'un LLM

- **Pré-entraînement** : lire une gigantesque bibliothèque (une grande partie d'Internet) et apprendre "qu'est-ce qui est vraisemblable après quoi". Très cher, une fois.
- **Fine-tuning** : un entraînement(propre) avec des conversations et du RLHF pour le rendre utile et poli.
- **Inférence** : l'utilisation à toi. Chaque fois que tu lui parles, le modèle **shift de droite à gauche**, prédit le token suivant, puis le suivant…

2.4 Pour bien savoir parler à un LLM (basics de prompting)

- **Prompt d'agent système** : la consigne de comportement ("Tu es un expert en fiche de paye") — guide donc la tonalité et le format attendu. - **Zero-shot** : aucune exemple, juste la consigne. - **Few-shot** : 1-3 exemples de réponse attendue dans le prompt tbgelle sortie. - **Hallucination** : le modèle te donne une réponse **sans broncher** plausible mais **fausse**. Les LLM ne "savent" pas, ils **génèrent ce qui est vraisemblable**. C'est pour ça que la vérification humaine est cruciale. - **Paramètres rapides** : - `temperature` (0 = très déterministe/précis ; 0,7 = créative au hasard). - Vocabulaire : smarter/fancier words = plus créatifs, mais plus de hallucination.

> **À retenir :** (1) attention = poids entre les mots ; (2) token ≈ ¾ de mot ; (3) un LLM "sait" au sens de "générer du textémonsembl cnc", il ne "sait pas" au sens des certitudes — d'oùs halluc des doutes.

Module 3 — Vibe coding : programmer avec les Vibes

3.1 C'est quoi ?

Terme popularisé en 2025 par Andrej Karpoby (ex-OpenAI). **Définition :** une manière de développer où tu **confies le plus heavy possible du travail de code à l'IA**, en parlant en langage naturel, et où tu **te concentres sur ce que tu veux construire** plutôt que sur la syntaxe.

> "Premier commit avec des vi et un code que tu ne comprend pas toutes les lignes — mais ça marche, et l'on itère." — l'esprit du vibe coding.

3.2 Vibe coding vs développement classique

Développement classique	Vibe coding
Tu écris chaque ligne à la main	Tu décris, l'IA écrit
Tu lis la doc pour chaque fonction	Tu demandes à l'IA de tout gouduguer
Test/pas à pas lent	Itération très rapide : générer exécuter corriger
Tu maîtrises chaque défaut de détail	Risque : code que tu ne comprends pas

3.3 Le workflow typique

```
  [Idée en langage naturel] → [Prompt à l'IA] → [Code généré]
         ↑                                            ↓
[Tu lis/testes] ← [Exécution] ← [Itération : "ça marche pas, change X"]
```

- Commence par la plus **grande description** du projet.
- Demande l'arborescence des fichiers.
- Genéfie module par module.
- **Exécute, teste** après chaque gros morceau.
- Corrige via des requêtes précises ("dans la partie exacte du module auth...").
- **Commit** régulièrement (git) pour garder des marches arrière.

3.4 Bonnes pratiques (à avoir en tête pour la classe et les stages)

- **Ne jamais exécuter du code que tu ne comprends pas** quand ça touche au réseau, aux cles API, ouModifie le système.
- **Soit la loop code review** : fais relire le code par l'IA ou un humain avant de le considérer validé.
- **Versionne** (git) — sinon une mauvaise génération peut tout casser sans possibilité de revenir en arrière.
- Mieux : utilise des **prompts itératants et tests** (l' agent de vibe coding fat ici lien avec agents).
- Garde à l'esprit les limites : dette technique, hallucination, biais, ne pas l'utiliser pour des systems d'usage critique sans surveillance.

> **À retenir :** (1) l'IA devient l'outil principal, le problème le focus ; (2) workflow = itérate Generate Execute Fix ; (3) la compétence clé devient savoir **exprimer tannant ce qu'on veut** et **vérifier** ce qui sort.

Module 4 — MCP (Model Context Protocol) : les connecteurs de l'IA

4.1 C'est quoi, en une phrase

Le MCP est un protocole ouvert (lancé par Anthropic fin 2024) qui permet à une IA (assistant, agent) de se connecter à des outils externes — exactement comme une "prise USB" standard pour donner des capacités aux IA.

4.2 Le problème qu'il règle

Sans MCP, pour donner l'accès à ton IA à : - tes Google Calendar - tes fichiers - ton repo GitHub - ta data de l'entreprise … il fallait coder à la main des **intégrations spécifiques** pour chaque outil et chaque modèle. Avec MCP, on écrit **une seule fois un serveur** qui expose des capacités, et n'importe quel client IA peut s'y brancher.

4.3 L'analogieConcrete

```
  Ton ordinateur = l'IA (le client MCP)
La prise USB / le port Ethernet = le protocole MCP
Le serveur MCP = un adapteur qui expose des capteurs spécifiques (Slack, GitHub, Postgres…)
```

4.4 Petite architecture

- **MCP Server** : le programme qui expose des **outils (tools)**, des **ressources** (contexte) et des **prompts** (routines prêtes à l'emploi). - **MCP Client** : l'agent ou l'IA qui s'y connecte. - **Transports** : comment voyagent les messages - `stdio` : via la ligne de commande (mode local, le plus courant en dev) - `SSE` / `HTTP` : via le réseau (mode distant) - **Outils (tools)** : des actions appelables que le modèle peut déclencher ("lire le fichier X", "créer une issue GitHub", "poster un message Slack").

4.5 Ou en rencontrerez-vous

En cours de v variabile traditionnels, sourcés dans : - **n8n** : le workflow que tu utilises expose un serveur NCP (voir tes sheets). - **Cursor / Claude Desktop / Copilot** : clients MCP. - **Intégrations** : GitHub, Slack, Google Drive, Notion, Postgres, MongoDB, etc.

```
  // Exemple ultra-simplifié de ce que "voit" un client MCP
{
  "tools": [
    { "name": "read_calendar", "description": "Lit les rendez-vous...", "inputSchema": {...} },
    { "name": "send_slack", "description": "Envoie un message...", "inputSchema": {...} }
  ],
  "resources": ["cron://meetings/..."],
  "prompts": ["préparer_le_rapport"]
}
```

Le modèle lit cette liste, et **choisce les outils au bon moment** là où l'IA *agit* plutôt que *répond* (lien vers Module 6).

4.6 Vocabulaire MCP

- **Étapes** : installation configuration (chemin du sda) appel - **Serveur local** : s'exécute sur ta machine (`npx some-mcp-server`) - **Diskorré** : "MCP connector" = tout adaptateur dont tu parles en classe - **Sans serveur** vs pas

À retenir : (1) MCP = prise standard pour donner des outils à une IA ; (2) un serveurMCP expose tools/ressources/prompts ; (3) stdio pour le local, SSE/RSS lلqroupé.

Module 5 — _CLI : la sacoche de survie en ligne de commande

Pourquoi en parler ? Parce que le vibe coding et les agents MCP **se commandent très largement en ligne de commande** (terminal). Savoir lire l'anglais de la CLI, une compétence de base pour suivre (npx, pip, git...).

5.1 Réflexes de lecture

```
  chemin@machine:~$ commande options/arguments
```

- `pwd` où suis-je ? (`/` = la racine) ; `ls` qu'y a-t-il ici ? - `cd Bureau` entrer ; `cd ..` remonter d'un niveau - `mkdir projet` créer un dossier ; `rm -r dossier` supprimer (danger) - `cp a b` copier ; `mv a b` déplacer/renommer - `cat depot.txt` afficher ; `grep "mot" fichier` chercher dedans

5.2 Outil indispensables pour l'IA

```
  # Installer / lancer des trucs
npx something-mcp-server        # lance un serveur MCP sans l'installer durablement
npm install   /   pip install   # installer des paquets
pip install mcp                 # la librairie python officielle du protocole MCP

# Git (versionnement)
git clone https://...               # copie un projet
git status                          # état de tes fichiers
git add . && git commit -m "fix"    # prendre une version
git push                            # mettre en ligne
```

5.3 3 réflexes pour ne pas se faire peur

- `help` : presque chaque outil répond à `--help` (ex. `npx --help`).
- **Ctrl+C** pour arrêter un processus qui tourne en boucle.
- **Environnement variables (` ennvironment env`)** : les clés d'API (ou comme `API_KEY`) sont souvent stockées tout plis de la CLI (via `.env`) — ne jamais les commitounovsky.

À retenir : (1) la CLI = ov-oserie de l'univers des agents IA en local ; (2) npx/npm/pip et git font 90% des besoins ; (3) `--help` et `Ctrl+C` = tes deux boutons de panique.

Module 6 — Agents et "agentic" : comprendre quand l'IA s'aggoit

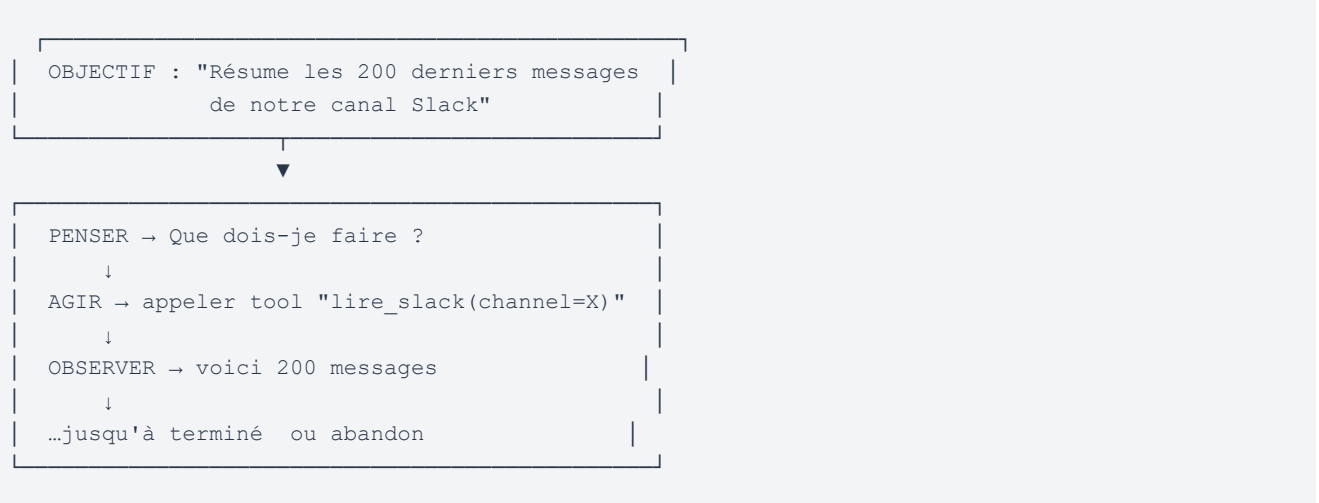
6.1 Rien d'autre réponse ≠ agement

One-shot (classique)	Agent
Tu poses, il répond, fin de l'histoire	Il agit avec un but précis, en plusieurs étapes, en utilisant des outils
"Quelle est la cap' de Paris ?" réponse	"Planifie-moi un roadtri de 3 jours en Alsace" il consulte météo, hôtels, cartes…

Une agent = un modèle qui peut :

- **Observer** (tools de l'environnement),
- **Réfléchir** (son propre texte de raisonnement),
- **Agir** (appel d'outils, de code, d'API),
- Répéter — dans **la** boucl(loop) …jusqu'à atteindre l'objectif.

6.2 Le schéma de la boucle d'agent la plus canonique



Ce schéma s'appelle **ReAct** (Reason + Act). Il mélange dans un même texte l'raisonnement (chains-of-thought) et l'action.

6.3 Quand on en parle

- Par t-œil : `multi-agent` / `socety of_agents` (plusieurs agents avec des rôles, qui coopèrent ou se challengent). - **agentic workflow** = les workflows ou l'IA pilote le séquencement des outils (des centaines aux agents). - **Prompt injection** = technique d'attaque qui détourne le comportement d'un agent ou d'un LLM avec un prompt malicieux (sécurité !).

6.4 Qui fait tourner ça ? (toolbox)

Outil territoire	Spécialité
n8n	Automatismes vis issu (workflow), parlé IA
LangChain / LangGraph	Chain/flex pour composer des agents en Python

Outil territoire	Spécialité
CrewAI / AutoGen	Multi-agents
Pydantic Agents	Le nouveau SDK light d'Anthropic (2025)

L'**agentic** en entreprise = autonome de bout en bout (choisir ses tools, naviguer, exécuter, se corriger) pç de main humaine.

À retenir : (1) difference :one-shot répondre vs agent agir ; (2) l'agent tourne dans une boucle Observer Penser Agir Observer ; (3) les "tools" c'est la matière première (le MCP est le connecteur) ; (4) la course : plus autonome, plus efficace… mais plus risque si on ne contrôle.

Module 7 — Théorie des jeux : l'IA qui strategic entre elle

(Version "2 minutes", juste ce qu'il faut pour situer les concepts en cours.)

7.1 Idées de base

- Un **jeu** = un ensemble de **joueurs**, qui chacun choisissent une **stratégie**, et reçoivent un **gain** (payoff). - Game **coopératif** (les joueurs peuvent s'entendez/contrats) vs **non coopératif** (chaque joueur joue pour lui). - **Zero-sum** : ce que gagne l'un, l'autre le perd (échecs). **Non zero-sum** : souvent, tout le monde peut grignoter (coo détenue entre concurrents).

7.2 Les 3 concepts qui reviennent tout le temps

- **Dilemme du prisonnier** — les deux joueurs ont intérêt à PENAL que l'autre, même si la coopération commune est meilleure. La métaphore des agents IA qui ne se font pas confiance.
- **Équilibre de Nash** — une situation où **personne ne peut améliorer son gain en changeant tout seul de stratégie**. L'équilibre "stable".
- **Forge / adversaire tour**: c'est l'idée de Reddit Optical — traines les autres parties qui elle aussi jeweiligen.

7.3 Dans les LLM concrètement

- **Traîne-par-RL multi-agents** : des agents s'affrontent (ex. un générateur d'arguments vs un critique) pour améliorer la qualité. - **Négociation entre agents** : des agents IA qui achètent/vent/transigent — pour savoir quand coopérer ou tromper. - **Systèmes de recommandation concurrentiels** : prix dynamiques, تايلآ d'enchères (ads, attention).

À retenir : (1) théorie des jeux = comment des décideurs rationnels anticipent les autres ; (2) Nash = personne ne regrette son choix pris isolé ; (3) les agents IA = l'endroit où ça fuse : coopérer, être en concurrence, se saturer.

Plan suggéré de 4 semaines (ordre + effort)

Semaine	Focus	À faire	Effort hebdo
---	---	---	---
S1	Fondations IA/ML + DL	Module 1 & 2 ; faire le mini-jeu "chat vs chien" mental ; comprendre attention et tokens	5-6h
2	Vibe coding en pratique	Module 3 ;prendre un petit projet (site perso), construire prompt-first	5-6h
3	Outillage pro (MCP + CLI + agents)	Modules 4, 5, 6 ; installer un serveur MCP local (npx/tutoriel officiel), brancher un agent simple	6-8h
4	Multis-agents + théorie des jeux	Module 7 ; lire qqs articles sur Nash & multi-agents ; réviser tout le glossaire	4-5h

Glossaire (A–Z des termes que tu croises en cours)

Terme	Définition courte et simple
---	---
Agent (IA)	Une IA configurée pour agir dans un but, en utilisant des outils et une boucle observateur décideur acteur.
Agentic	Approche où l'IA pilote elle-même un workflow d'outils de bout en bout (vs une simple demande-réponse).
AGI	Intelligence générale artificielle : une IA qui saurail de comprendre/apprendre n'importe quelle tâche intellectuelle (pas encore d'aujourd'hui).
Algorithme	Suite d'instructions précise pour résoudre un problème. La "recette" du modèle.
API (Application Programming Interface)	Un contrat technique qui permet à deux programmes de se causer entre eux (requêtes réponses structurées).
Attention (mécanisme)	Dans un transformer : pour chaque mot, la poids des autres mots du texte pour comprendre son sens.
Backpropagation	La technique qui remonte l'erreur du modèle jusqu'à ses poids pour les ajuster (après la phase d'erreur).
Batch	Un sous-ensemble de données traité en_mmune pendant l'entraînement.
Chatbot	Un système d'IA conçu pour discuter avec un humain.
CLI (Command-Line Interface)	L'interface en ligne de commande : on tape des commandes au lieu de cliquer.
Corpus (de données)	L'ensemble du texte/données de départ sur lequel on entraîne.
Dataset	Le jeu de données (structuré ou non) avec entrée[s] et parfois la réponse attendue.
Deep Learning (DL)	Le ML avec des réseaux de neurones profonds (plusieurs couches).
Descente de gradient	L'algorithme qui ajuste les poids en direction de la pente la plus descendante (minimiser l'erreur).
Embedding	Une suite de nombres / un "code" qui résume le sens d'un mot ou d'un texte.
Epochs (époques)	Un tour complet d'entraînement sur tout le jeu de données. On en fait plusieurs.
Entraînement (training)	La phase où le modèle apprend en ajustant ses poids.
Éthique de l'IA	Les questions d'équité, de transparence, de biais, de vie privée, de responsabilité.
Fine-tuning	Ré-entraînement d'un modèle existant sur des données spécifiques pour le spécialisé/guider.
Framework	Une bibliothèque prête à l'emploi pour construire des modèles (PyTorch, TensorFlow, NumPy).
Gradient	La direction et la pente pour diminuer de manière maximale l'erreur (la "pente à descendre").

Terme	Définition courte et simple
Hallucination	Réponse générée par un LLM qui est plausible mais fallacieuse. Risque majeur des LLM
Hyperparamètre	Les réglages qu'on choisit AVANT d'entraîner (nb de couches, taux d'apprendre, température…).
IA (Intelligence Artificielle)	Le champ global : faire raisonner les machines.
IA conversationnelle / chatbot	Un LLM adapté pour converser (ChatGPT, Claude…).
IA générative	Des modèles capables de produire du contenu nouveau (texte, image, image, musique, code).
Inférence	L'utilisation du modèle déjà entraîné (les requêtes en ligne temporaire).
LangChain	La bibliothèque la plus connue pour composer des agents LLM en Python/JS (chaines d'appels d'outils).
LLM (Large Language Model)	Grand modèle de langue : des milliards de poids entraînés sur d'énormes volumes de texte.
MCP (Model Context Protocol)	Protocole ouvert qui connecte des outils/API externes à un client IA (serveur client, stdio/SSE/HTTP).
Modèle	Une fonction mathématique paramétrée qui transforme une entrée en une sortie probable.
Modèle multioal	Un modèle qui comprend plusieurs types de données (texte + image + audio + vidéo)
Neurone artificiel	L'unité de base d'un réseau : diversi en entrée, 1 output, multipliée par des poids.
NLP (Natural Language Processing)	Le sous-domaine du traitement automatique du langagement naturel (texte & parole).
Overfitting	Sur-apprentissage : le modèle apprend par cœur les exemples au lieu de généraliser.
Paramètres / poids	Les nombres internes du modèle que l'entraînement ajuste (des milliards).
Prompt	Le texte (avec consignes) que l'on donne à un LLM pour obtenir une réponse.
Prompt injection	Attaque par le prompt : insérer des consignes cachées pour détourner le comportement du modèle.
Prompting	L'art de formuler des prompts efficaces pour obtenir le résultat voulu.
PyTorch / TensorFlow	Les deux grands frameworks de deep learning.
RAG (Retrieval-Augmented Generation)	Récupérer de la doc externe avant de générer pour réduire les hallucinations.
Réseau de neurones	Un ensemble de neurones organisés en couches connectées qui apprennent sur des données.
Reward / récompense	La note donnée à l'agent en RL pour ses actions (positive ou négative).
RL (Reinforcement Learning)	L'apprentissage plein et عساك par essais-erreurs avec récompenses (jeux, contrôles…).

Terme	Définition courte et simple
RLHF	D'apprentissage du modèle par des feedbacks humains (notation des réponses).
Serveur MCP	L'adaptateur qui expose tools/ressources/prompts à un client IA. Gratuit plus rarement un serveur.
SOC [Society of Agents]	Organisation de plusieurs agents chacun avec un rôle (squad, contrôle…).
Supervised Learning	L'apprentissage avec des étiquettes/réponses d'exemple (transla, classif).
Systémique / unsupervised	L'apprentissage sans étiquettes : trouver des groupes ou des anomalies naturellement.
Test set / validation set	Sous-jeux de données pour évaluer le modèle sans le montrer pendant l'entraînement.
Token	Unité de texte (mot ou fragment de mot) que le LLM calcule (≈ ¾ de mot).
Temperature	Un réglage qui contrôle la créativité (faible = déterministe, haut = aléatoire).
Tool calling	Le fait qu'un LLM puisse appeler un outil/API/fonction lui-même pour compléter sa réponse.
Transformer	L'architecture de lesmodèles de langue moderne, basée sur l'attention.
Training data	Les données d'entraînement.
Underfitting	Sous-apprentissage : le modèle trop simple pour apprendre les motifs.
Vector database (base vecteurielle)	Base de données qui conserve des embeddings pour de, de similarités.
Vibe coding	Développer guidé par les suggestions de l'IA générative en langage naturel (terminé popularisé 2025).
Workflow	Une séquence d'étapes (souvent un automate) qu'un système execute de bout en bout.
Zero-shot / Few-shot	Demander une réponse sans exemple (zero) vs avec 1-3 exemples dans le prompt (few).

Ressources gratuites (et débutants-friendly)

- **"AI For Everyone"** (Coursera, Andrew Ng) — cours grand public, idéal pour débuter sans maths.
- **DeepLearning.AI — Short Courses** — mini-cours pratiques (gratuits, 1-2 h chacun).
- **3Blue1Brown** — vidéos visuelles sur le ML et le deep learning (playlists "Neural networks").
- **Hugging Face "LLM University"** — gratuit, très pédagogique sur les LLM.
- **docs.modelcontextprotocol.io** — la doc officielle du MCP (section "Quickstart" pour brancher un serveur).
- **"Building Effective Agents"** (Anthropic engineering blog) — le bon texte sur les agents.
- **n8n docs** + vos workflows, pour voir agents/MCP en pratique.

Guide préparé pour un public francophone débutant · dernière mise à jour : octobre 2026. "Rien ne vaut le fait de tomber sur un concept en cours et de revenir le chercher ici pour comprendre."